

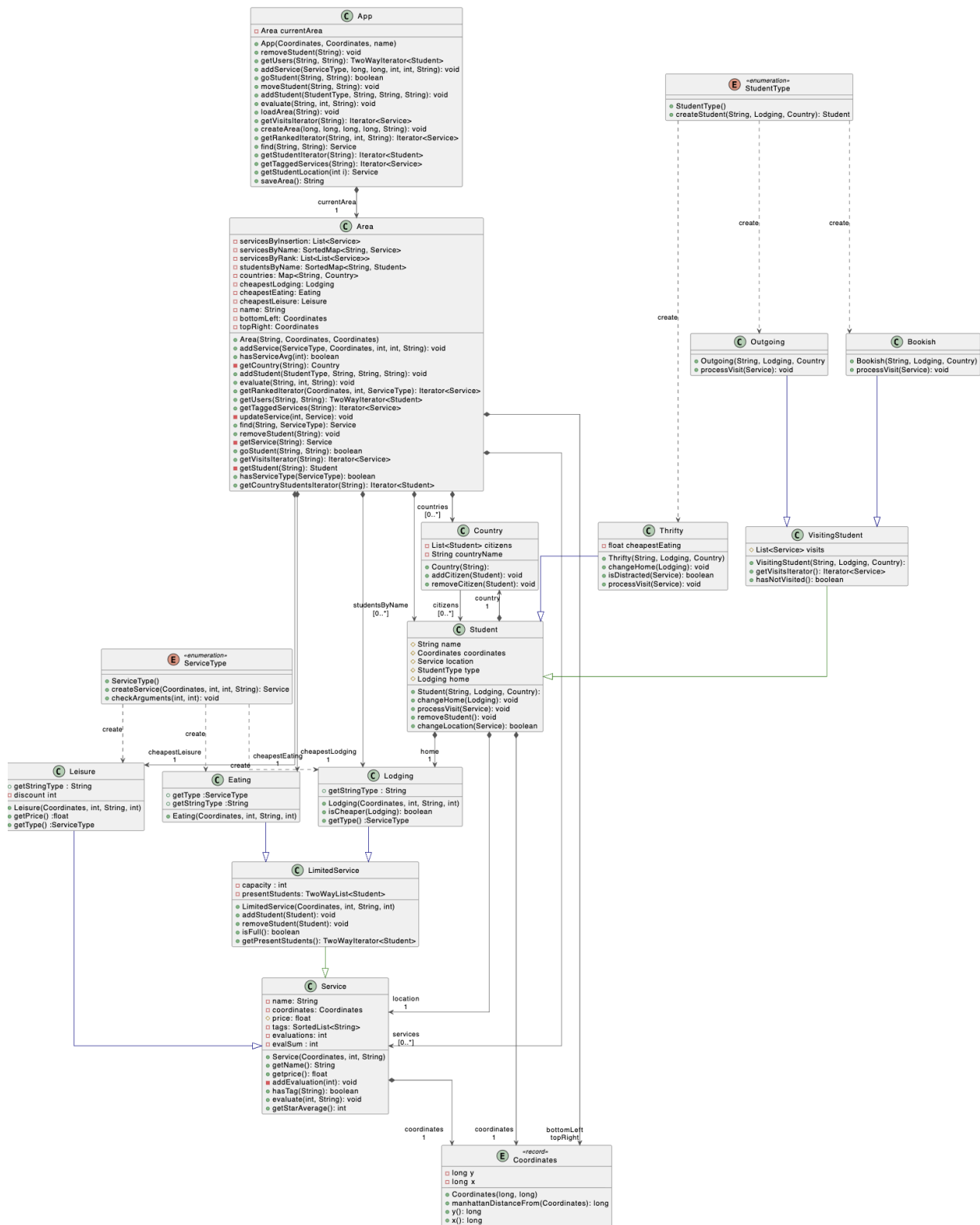
1º Relatório AED

HomeAway From Home

Guilherme Santos | 65443

Bernardo Lobão | 68022

Comando / Método	Classes / Variáveis envolvidas	Melhor caso (tempo)	Pior caso (tempo)	Justificação / Cenário
<code>isUndefined()</code>	<code>App.currentArea</code>	$O(1)$	$O(1)$	Apenas checka se <code>currentArea</code> é <code>null</code>
<code>convertCoordinates(x, y)</code>	<code>Coordinates</code>	$O(1)$	$O(1)$	Cria um novo objeto <code>Coordinates</code>
<code>saveArea()</code>	<code>App.currentArea</code> , <code>Area.servicesList</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Serializa a área; n = número total de objetos
<code>loadArea(areaName)</code>	<code>App.currentArea</code> , <code>Area</code>	$O(1)$	$O(n)$	Desserializa todos os objetos da área
<code>isInvalidArea(xMin, xMax, yMin, yMax)</code>	—	$O(1)$	$O(1)$	Apenas comparações numéricas
<code>existsFile(fileName)</code>	—	$O(1)$	$O(1)$	Chamada direta ao sistema de ficheiros
<code>getFileName(s)</code>	String de entrada	$O(n)$	$O(n)$	Percorre a string para substituir espaços
<code>createArea(...)</code>	<code>App.currentArea</code> , <code>Coordinates</code>	$O(1)$	$O(n)$	Criação da área é $O(1)$; salvar área anterior depende do tamanho
<code>addService(type, coordinates, price, value, name)</code>	<code>App.currentArea</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Depende da inserção na lista/AVL ou verificação de duplicados
<code>getServicesIterator()</code>	<code>App.currentArea</code> , <code>Area.servicesList</code>	$O(1)$	$O(1)$	Apenas cria e retorna iterador
<code>getAreaName()</code>	<code>App.currentArea</code>	$O(1)$	$O(1)$	Acesso direto ao nome da área
<code>getServiceName(serviceName)</code>	<code>App.currentArea</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Busca pelo nome do serviço
<code>getStudentName(studentName)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Busca pelo nome do estudante
<code>addStudent(studentType, name, country, home)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Inserção simples ou verificação de duplicados
<code>removeStudent(name)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Remoção direta ou busca sequencial
<code>getAllStudentsIterator()</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(1)$	Criação de iterador
<code>getCountryStudentsIterator(country)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Filtra estudantes por país
<code>getStudentLocation(studentName)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Acesso direto ou busca sequencial pelo estudante
<code>goStudent(name, location)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Procura estudante e destino
<code>setHome(student, lodging)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Busca estudante e atualiza residência
<code>getUsers(order, service)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Cria iterador bidirecional com lista temporária
<code>getVisitsIterator(student)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code> , <code>Student.visits</code>	$O(1)$	$O(n)$	Depende do número de visitas registradas
<code>evaluate(service, stars, tags)</code>	<code>App.currentArea</code> , <code>Area.servicesList</code> , <code>Service.evaluations</code>	$O(1)$	$O(n)$	Atualiza média de avaliações e adiciona tags
<code>getRankingServices()</code>	<code>App.currentArea</code> , <code>Area.servicesList</code> , <code>Service</code>	$O(1)$	$O(n)$	Ordena serviços por avaliação
<code>getTaggedServices(tag)</code>	<code>App.currentArea</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Percorre lista filtrando serviços com a tag
<code>getRankedIterator(studentCoordinates, stars, serviceType)</code>	<code>App.currentArea</code> , <code>Area.servicesList</code> , <code>Service</code>	$O(1)$	$O(n)$	Filtra serviços por tipo, distância e média mínima
<code>getStudentCoordinates(studentName)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code>	$O(1)$	$O(n)$	Busca coordenadas do estudante
<code>hasServiceType(serviceType)</code>	<code>App.currentArea</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Verifica se existe serviço de determinado tipo
<code>hasServiceAvg(stars)</code>	<code>App.currentArea</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Verifica se existe serviço com média mínima
<code>find(studentName, serviceType)</code>	<code>App.currentArea</code> , <code>Area.studentsList</code> , <code>Area.servicesList</code>	$O(1)$	$O(n)$	Busca serviço com base no tipo e estudante



Complexidade espacial: $O(3n+2m+c+0.5n*m+n*0.1m)$

Sendo que:

N - representa o número de serviços no sistema

M - representa o número de estudantes no sistema

C - representa o número de países no sistema

$n*0.1m$ - representa o número esperado de estudantes em cada serviço

$0.5n*m$ - representa o número de serviços que cada estudante visitou

Breve Discussão:

Acreditamos que foi nos possível desenvolver uma boa execução do projeto. Usando *SepChainHashTable's* e *AVLSortedMap's* quando necessário e quando a implementação tornaria o programa mais eficiente.

O uso de uma lista dentro de uma lista para guardar os serviços de ordem de *rank* tornou a execução do projeto bastante eficiente principalmente nos comandos *find* e *ranked*.

No geral, o grupo achou que encontrou soluções que satisfazem os problemas do projeto. Usou-se estruturas de dados que permitem a execução bastante rápida do programa.

Para ter como referência deixar-se-á um anexo com os tempos dos 18 testes corridos na máquina de um membro do grupo.

Para uma possível melhoria no trabalho discutimos a implementação de uma ED *SepChainHashTable<String, Service> ServicesByTag* para melhorar a eficácia do comando *tags*.

✓ Tests	587 ms
✓ test01	113 ms
✓ test02	32 ms
✓ test03	26 ms
✓ test04	11 ms
✓ test05	10 ms
✓ test06	11 ms
✓ test07	9 ms
✓ test08	11 ms
✓ test09	16 ms
✓ test10	14 ms
✓ test11	9 ms
✓ test12	9 ms
✓ test13	9 ms
✓ test14	35 ms
✓ test15	14 ms
✓ test16	58 ms
✓ test17	102 ms
✓ test18	98 ms

(Nota: A máquina referida é um Macbook air M2)